

Vektoripaja

Paperinen työpaketti

Low-poly 3D-mallinnin Windows-työpöytäsovelluksena (Python): Inkscapen SVG → 3D-malli → .obj.

Tekoälyavusteinen työtapa: Microsoft 365 Copilot suunnittelee ja auttaa Tiedosto-kaistalla. GitHub Copilot avustaa VS Codessa.

Vk 40/2026–vk 9/2027 · syysloma vk 42 · joululoma 18.12.–10.1. · talviloma vk 8 · luovutus pe 5.3.2027

Tämä paketti on aikataulu ja tarkistuslista tilanteisiin, joissa sivusto ei ole auki. Projektipäiväkirja kirjoitetaan sivustolla ja viedään repositoryn project-docs-kansioon. Rasti tässä vihossa ei ole palautus: työ on aina Git-repositoryssa.

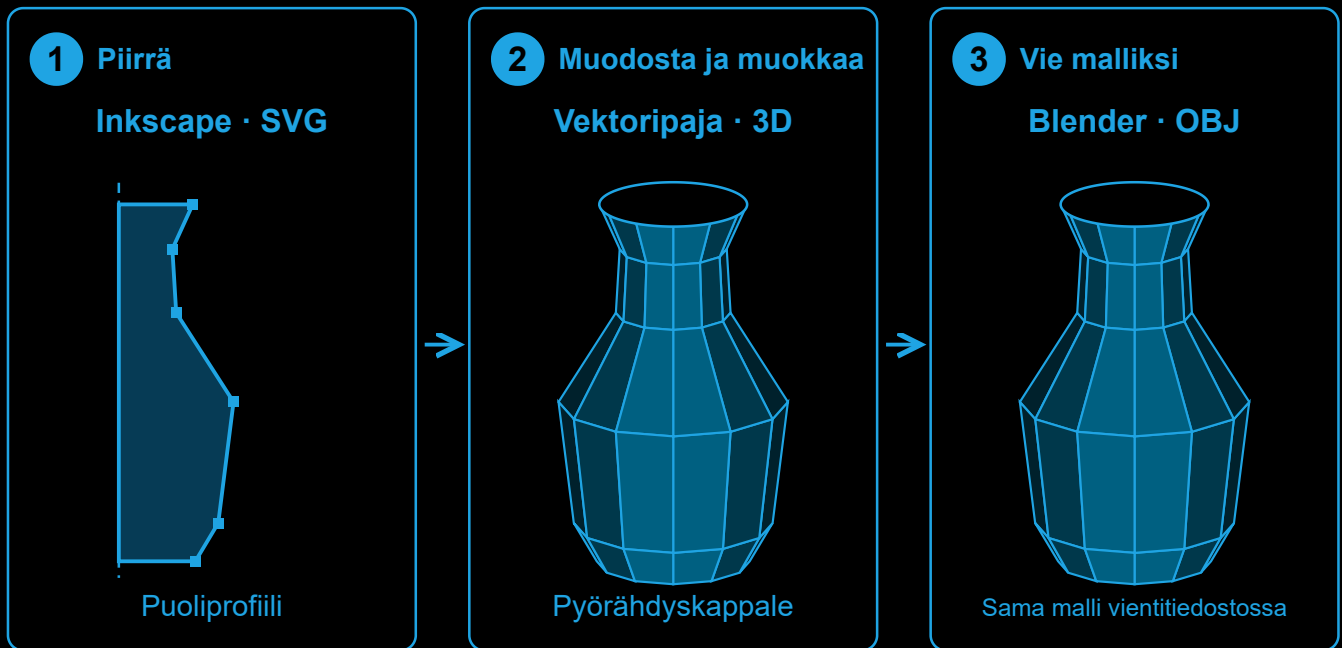
Repository on julkinen. Sinne ei laiteta henkilötietoja, testaajien nimiä eikä itsearviointia. Ne lähetetään ohjaajalle Teamsissa.

Viikkopalaveri ohjaajan kanssa on maanantaina tai tiistaina. Päivä sovitaan viikolla 40.

Mitä rakennat ja miten etenet

Piirrät Inkscapeissa. Vektoripaja tekee piirroksesta 3D-kappaleita, joita käyttäjä voi muokata ja tallentaa.

Valmis malli viedään Blenderiin OBJ-tiedostona.



1. Piirrä lähtökuva Inkscapeissa ja tallenna se SVG-tiedostona.

2. Avaa kuva Vektoripajassa. Muodosta 3D-kappaleet, muokkaa osia ja tallenna työ.

3. Vie valmis malli OBJ-tiedostona Blenderiin tai toiseen 3D-ohjelmaan.

Projektin viisi vaihetta

1. **Valmistellaan työkalut ja työtap.** Viikot 40–41: työkalut ja kuutioharjoitus. Opit tekemään, testaamaan ja julkaisemaan. Syysloma viikolla 42.

2. Rakennetaan piirroksesta 3D-malli. Viikot 43–47: piirroksen tuonti, osat, muodot ja muokkaaminen. Sovelluksen perustoiminnot syntyvät.

3. Tehdään ensimmäinen käyttökelpoinen versio.

Viikot 48–51: kiertopiste, vienti, tallennus ja asiakkaiden kokeilu. Joululoma viikoilla 52–1.

4. Parannetaan asiakkaiden palautteen perusteella.

Viikot 2–5: korjaukset ja sovitut jatkotoiminnot. Palaute ohjaa seuraavia muutoksia.

5. Julkaisu ja näyttö. Viikot 6–9: julkaisutesti, v1.0 ja esittely. Talviloma viikolla 8, näyttö viikolla 9.

Viikon työvaihe on työohje. Kun rakennat muutosta sovellukseen, tee siitä GitHub-issue ja käytä työsykliä. Kirjaa testitulokset issueen ja viikon yhteenveto päiväkirjaan.

Aikataulu

Vko	Pvm	Viikon aihe	Vaihe
40	28.9.– 2.10.	Aloitus: työkalut ja projektin rajaus	1
41	5.–9.10.	Harjoitus: pyörivä kuutio julki	1
42	12.– 16.10.	Syysloma – ei projektityötä	–
43	19.– 23.10.	Piirroksen tuonti (SVG)	2
44	26.– 30.10.	Piirroksen ryhmistä mallin osiksi	2
45	2.–6.11.	Profiilista pyörähdyskappale (revolve)	2
46	9.–13.11.	Viivasta putki ja mallin näkymät	2
47	16.– 20.11.	Osien siirto, kierto ja skaalaus	2
48	23.– 27.11.	Kiertopisteen säätäminen (pivot)	3
49	30.11.– 4.12.	Mallin vienti Blenderiin (OBJ)	3

50	7.–11.12.	Tallennus ja ensimmäinen toimiva versio	3
51	14.–17.12.	Asiakkaat kokeilevat ensimmäistä versiota	3
52	21.–25.12.	Joululoma – ei projektityötä	–
53	28.12.–1.1.	Joululoma – ei projektityötä	–
1	4.–8.1.	Joululoma – ei projektityötä	–
2	11.–15.1.	Paluu ja parannusten järjestys	4
3	18.–22.1.	Muokatun piirroksen päivittäminen	4
4	25.–29.1.	Puutteet ja sovitut parannukset	4
5	1.–5.2.	Selkeät säätimet ja näppäimistökäyttö	4
6	8.–12.2.	Julkaisun kokeilu toisella koneella	5
7	15.–19.2.	Valmis sovellus v1.0	5
8	22.–26.2.	Talviloma – ei projektityötä	–
9	1.–5.3.	Projektin esittely ja luovutus	5

Palautus viimeistään pe 5.3.2027. Sivusto: tehtävien tarkat ohjeet, toteutusavut ja projektipäiväkirja.

Vaihe 1 – Valmistellaan työkalut ja työtap

Vko 40 · 28.9.–2.10. – Aloitus: työkalut ja projektin rajaus

Vektoripajassa Inkscapella tehty piirros muuttuu muokattavaksi 3D-malliksi, jonka voi viedä Blenderiin. Tällä viikolla laitat työkalut ja oman repositoryn kuntoon sekä rajaat ensimmäisen toimivan version eli MVP:n. Näin seuraavien viikkojen toteutuksella on selkeä tavoite ja paikka, jossa muutokset säilyvät.

Työkalut ovat käytössä, projektilla on oma repository ja ensimmäinen versio on rajattu.

Työvaihe 1 / 4 · Valmistele työkalut

- ☐ Sovi aloitus. Sovi Teamsissa ohjaajan kanssa tekijänimi ja viikkopalaverin päivä, maanantai tai tiistai. Tee tämä ennen Gitin asetuksia.
- ☐ Tarkista työkalut. Tarkista VS Code, Git, Inkscape ja Blender. Asenna vain puuttuvat. Aja VS Coden terminaalissa `git --version`. Node.js:ää ei tarvita.
- ☐ Tarkista Python. Aja `python --version`. Jos versio ei ole Python 3.13, asenna python.org-sivulta Windows installer (64-bit). Valitse asennuksessa Add python.exe to PATH. Katso kuvaohje.

- Varmista pip. Sulje VS Code ja avaa se uudelleen. Aja `python -m pip --version`. Lopussa pitää lukea (python 3.13). Jos versio poikkeaa, lähetä tuloste ohjaajalle Teamsissa.
- Tarkista Python-laajennus. Avaa VS Codessa Extensions ja hae Python, julkaisija Microsoft. Uninstall tarkoittaa, että se on asennettu. Muuten valitse Install. Laajennuksella luodaan ensi viikolla virtuaaliympäristö eli projektin oma kirjastokansio.
- Vaihda teemat. Valitse VS Codessa Dark High Contrast ja GitHubin asetuksissa Dark high contrast.
- Kirjaa lähtötilanne. Kirjaa työkalujen versiot README:hen. Python-tulkki ajaa Python-koodin; pip asentaa kirjastot. Seuraavalla viikolla luot projektin virtuaaliympäristön (.venv).

Valmis kun: Työkalut käynnistyvät, Pythonin ja pipin versiot näkyvät ja teema on vaihdettu.

Tallenna työnäyte: Versiot README:hen ja sovittu tekijänimi suunnitelmaan.

Työvaihe 2 / 4 · Perusta projektin repository

- Aseta Git. Aseta Gitin nimeksi sovittu tekijänimi. Aseta sähköpostiksi GitHubin noreply-osoite. Näin omaa sähköpostiosoitettasi ei julkaista. Katso kuvaohje alta.

- ☐ Luo repository. Luo GitHubissa julkinen repository nimellä vektoripaja. Valitse Add a README file. README on repositoryn etusivun ohje.
- ☐ Kloonaa repository. Kloonaa repository VS Codeen. Kloonaus tekee siitä kopion omalle koneellesi.
- ☐ Lisää pohjat. Lisää kolme pohjatiedostoa ensi viikkoa varten: tilatiedosto PROJEKTIN-TILA.md, tehtäväkorttipohja ja .github/copilot-instructions.md. Pohjat ovat alla kopioitavina.
- ☐ Jaa repository. Lisää ohjaaja Collaboratoriksi. Collaborator on henkilö, joka näkee repositoryn ja voi kommentoida sitä.

Valmis kun: Repository on kloonattu VS Codeen, pohjat ovat mukana ja ohjaaja pääsee katsomaan työtä.

Tallenna työnäyte: Repositoryn osoite päiväkirjaan. Pohjat GitHubiin commitilla ja pushilla.

Työvaihe 3 / 4 · Rajaa ensimmäinen versio

- ☐ Kirjoita MVP. Täytä tällä sivulla suunnitelman MVP omin sanoin -kenttä: mikä kuuluu pakolliseen ytimeen ja miksi muut asiat odottavat.
- ☐ Lataa suunnitelma ja tallenna se tiedostoksi project-docs/suunnitelma.md. Perustele omin sanoin. Kirjaa Copilotin mahdollinen apu AI-lokiin.

- Tee commit ja push. Tallenna muutokset commitiksi. Commit on yksi nimetty muutos. Lähetä se GitHubiin pushilla.
- Lähetä kysymyslista. Kirjoita vähintään kolme kysymystä asiakkaille. Käytä pohjaa alta. Lähetä kysymykset Teamsissa. Vastaukset käydään läpi ensi viikon palaverissa.

Valmis kun: Oma MVP-kuvaus on GitHubissa ja kysymyslista on lähetetty asiakkaille.

Tallenna työnäyte: MVP-kuvaus suunnitelmaan. Vie suunnitelma project-docs/suunnitelma.md-tiedostoksi; kirjaa kysymykset project-docs/kysymykset.md-tiedostoon.

Työvaihe 4 / 4 · Mittaa agentin krediittikulutus

- Katso krediitit ennen mittausta. Avaa GitHubin käyttönäkymä. Käyttönäkymä on sivu, jolla krediittien kulutus näkyy. Kirjaa, montako krediittiä on käytetty.
- Avaa GitHub Copilot. Avaa VS Codessa GitHub Copilotin chat.
- Vaihda agenttitilaan. Valitse chatin tilaksi Agent. Agenttitila muokkaa tiedostoja ja kuluttaa krediittejä.
- Lähetä mittauksen viestipohja. Viestipohja on valmis viesti, jonka kopioit. Kopioi mittauksen viestipohja alta ja lähetä se. Hylkää ehdotettu muutos, jos se koskee muita tiedostoja kuin README:tä.

□ Kirjaa hinta. Avaa käyttönäkymä uudelleen. Hinta on nykyinen luku miinus aiempi luku. Kirjaa hinta päiväkirjaan. Jos luku ei ole muuttunut, kirjaa "odottaa käyttönäkymää" ja katso huomenna.

Valmis kun: Päiväkirjassa on kulutuksen muutos tai merkintä odottaa käyttönäkymää.

Tallenna työnäyte: Mittauksen alku- ja loppulukema sekä erotus tämän viikon päiväkirjaan.

Valmis kun: Ohjaaja näkee repositoryn, suunnitelma.md on GitHubissa, kysymyslista on lähetetty, ja päiväkirjassa on agenttipyyntö hinta tai merkintä "odottaa käyttönäkymää".

Työnäyte Git-repositoryyn ennen rastia: repositoryn osoite, project-docs/suunnitelma.md GitHubissa ja agenttipyyntö hinta päiväkirjassa.

Vko 41 · 5.–9.10. – Harjoitus: pyörivä kuutio julki

Ennen varsinaisia mallinnustoimintoja harjoittelet koko matkan tehtäväkortista testattuun ja ladattavaan sovellukseen. Pyörivä kuutio on pieni harjoitus, jolla varmistat, että kehitysympäristö, testit ja Windows-julkaisu toimivat yhdessä. Samaa työsykliä käytät myöhemmin Vektoripajan jokaisen ominaisuuden rakentamiseen.

Windowsissa käynnistyy ladattu harjoitussovellus, jossa kuutio pyörii.

Työvaihe 1 / 4 · Valmistele harjoitussovellus

- ☐ Käy viikkopalaveri. Käy läpi asiakkaiden vastaukset kysymyslistaasi. Kirjaa ne tiedostoon `project-docs/kysymykset.md`. Viikkopalaveri on viikon tapaaminen ohjaajan kanssa.
- ☐ Päätä ensin. Lue tekninen ehdotus toimeksiannosta. Kirjaa suunnitelmaan, hyväksytkö sen. Perustele päätös pakollisen ytimen toiminnoilla.
- ☐ Kirjaa käyttöliittymävaatimus. Kirjoita suunnitelmaan, millainen sovelluksesi ulkoasun pitää olla: taustaväri, tekstin väri, tekstin koko ja painikkeiden koko. Kortin #1 hyväksymiskriteeri tulee tästä.
- ☐ Pura pohja repositoryyn. Lataa alta vektoripaja-pohja.zip. Napsauta sitä hiiren oikealla painikkeella. Valitse Pura kaikki. Valitse kohteeksi repositorysi kansio. Pohjassa ovat valmiina julkaisun ja itsetestin tiedostot. Mukana on myös lista kirjastoista versioineen. Sen nimi on `requirements.txt`.
- ☐ Luo virtuaaliympäristö itse. Avaa repositoryn kansio VS Codessa. Paina Ctrl+Shift+P. Valitse Python: Create Environment. Valitse sitten Venv. Valitse lopuksi Python 3.13. Jos VS Code tarjoaa

kirjastojen asennusta, älä valitse listasta mitään.

Valitse OK.

☐ Asennat kirjastot itse seuraavassa askeleessa.

Katso kuvaohje alta.

☐ Asenna kirjastot itse. Avaa uusi terminaali.

Tarkista, että terminaalin rivin alussa lukee (.venv).

Aja `pip install -r requirements.txt`. Aja sitten `python tarkista_ymparisto.py`. Rivien alussa pitää lukea OK. Viimeisellä rivillä lukee Ympäristö on kunnossa.

☐ Kirjaa kirjastot README:hen. Kirjoita README:hen kirjastojen nimet ja lisenssit. Lisenssin näet komennolla `pip show`. Kirjoita komennon perään kirjaston nimi, esimerkiksi `pip show PySide6`.

Valmis kun: Ympäristötarkistus näyttää OK ja tekninen ehdotus on arvioitu omin sanoin.

Tallenna työnäyte: Päätökset suunnitelmaan; asiakkaiden vastaukset kysymykset.md-tiedostoon ja lisenssit README:hen.

Työvaihe 2 / 4 · Määrittele kuution testi

☐ Kirjoita testi 1 ennen koodia. Testi 1: kiertokulma.

☐ Funktio `kiertokulma(aika, nopeus)` saa ajan 0 ja nopeuden 0,5. Mitä odotat? Entä kun aika on 3? Kirjaa vastaukset ennen ajoa.

Valmis kun: Testin 1 odotetut arvot on kirjoitettu ennen toteutusta.

Tallenna työnäyte: Testi tests/test_kierto.py-tiedostoon ja odotetut tulokset GitHub-issueen.

Työvaihe 3 / 4 · Rakenna pyörivä kuutio

- ☐ Tee kortti #1 työsyklillä. Kortti: kuutio pyörii sovelluksen ikkunassa.
- ☐ Vertaa Copilotin korttia toteutusavun mallikorttiin.

Valmis kun: Sovellus näyttää pyörivän kuution ja testi 1 menee läpi.

Tallenna työnäyte: Koodi ja testi GitHubiin; testiajon tulos kuution issueen.

Työvaihe 4 / 4 · Julkaise harjoitussovellus

- ☐ Tee kortti #2 työsyklillä. Kortti: sovellus julkaistaan releasena. Release on GitHubin julkaisusivu, josta valmiin version voi ladata zip-tiedostona. Julkaisu alkaa nimitystä versiosta Gitissä. Sitä kutsutaan tagiksi. Katso toteutusavusta, miten julkaiset.
- ☐ Tee testi 2 puretusta kansiota. Testi 2: julkaisu. Lataa zip releasesta. Pura se valitsemalla Pura kaikki. Käynnistä Vektoripaja.exe puretusta kansiota, älä VS Codesta. Kuution pitää pyöriä.

Valmis kun: Ladatusta zipistä käynnistetty Vektoripaja.exe näyttää pyörivän kuution.

Tallenna työnäyte: Release v0.0.41 GitHubiin ja testin 2 tulos julkaisuissueen.

Valmis kun: Releasen zipistä purettu sovellus käynnistyy, ja kuutio pyörii. Actionsin ajo on vihreä, eli itsetesti meni läpi. pytest näyttää, että testi 1 menee läpi. Issuet #1 ja #2 on suljettu commitilla.

Työnäyte Git-repositoryyn ennen rastia: releasen v0.0.41 osoite, issuet #1 ja #2 suljettuina sekä testien 1 ja 2 tulokset issueissa.

Vko 42 · 12.–16.10. – Syysloma

Ei projektityötä eikä korvaavia tehtäviä. Jatka viikolla 43 tilatiedoston Seuraavana-kohdasta.

Vaihe 2 – Rakennetaan piirroksesta 3D-malli

Vko 43 · 19.–23.10. – Piirroksen tuonti (SVG)

Vektoripajan lähtöaineisto on käyttäjän Inkscapeissa tekemä SVG-piirros. Tällä viikolla tuot sen polut sovellukseen ja tarkistat, että myös virheelliset tiedostot käsitellään hallitusti. Tuonti luo pohjan osien rakenteelle ja myöhemmille 3D-muodoille.

Käyttäjä voi avata oman SVG-piirroksensa Vektoripajassa.

Työvaihe 1 / 5 · Piirrä tuonnin testiaineisto

- ☐ Piirrä testitiedosto. Tee Inkscapeissa nimetyt layerit eli tasot, kolme sisäkkäistä ryhmää ja kolme polkua.
- ☐ Katso kuvaohje alta. Tallenna tiedosto kansioon `testiaineisto/`.

Valmis kun: SVG:ssä on nimetyt tasot, kolme sisäkkäistä ryhmää ja kolme polkua.

Tallenna työnäyte: SVG-tiedosto repositoryn `testiaineisto`-kansioon.

Työvaihe 2 / 5 · Määrittele tuonnin testit

- ☐ Kirjoita testit 3–5 ensin. Testi 3: kolme polkua. Tiedostossa on kolme polkua. Testi 4: haitallinen SVG. SVG:ssä on `<script>`-elementti. Testi 5: väärä

tiedosto. Tiedosto ei ole SVG. Kirjaa jokaiselle, mitä odotat.

□ Lisää havaintopohja. Tallenna havaintopohja tiedostoon `.github/ISSUE_TEMPLATE/havainto.md`. Kun huomaat virheen, tee siitä havaintoissue: mitä odotit, mitä tapahtui ja miten virheen saa toistettua.

Valmis kun: Testien 3–5 syötteet ja odotetut tulokset on määritelty ennen toteutusta.

Tallenna työnäyte: Testit tests-kansioon ja havaintopohja `.github/ISSUE_TEMPLATE`-kansioon.

Työvaihe 3 / 5 · Suunnittele sovelluksen rakenne

□ Kirjaa rakenne. Kirjaa suunnitelmaan kansiorakenne ja moduulien rajat. Moduuli on kansio tai tiedosto, jolla on yksi vastuu. Päivitä sama rakenne tiedostoon `copilot-instructions.md`.

□ Sovi dokumentointitapa. Sovi palaverissa, miten ohjelma dokumentoidaan: README ja käyttöohje. Kirjaa sovittu tapa suunnitelmaan.

Valmis kun: Kansiorakenne, moduulien vastuut ja sovittu dokumentointitapa on kirjattu.

Tallenna työnäyte: Päätökset suunnitelmaan; päivitetty suunnitelma GitHubiin.

Työvaihe 4 / 5 · Rakenna SVG-tuonti

☐ Toteuta työsyklillä. Kirjoita testit Täydennys-kaistalla kommentista.

☐ Toteuta tuonti Tiedosto-kaistalla, tiedosto kerrallaan.

Valmis kun: Oman SVG:n polut näkyvät sovelluksessa ja testit 3–5 menevät läpi.

Tallenna työnäyte: Tuonnin koodi ja testit GitHubiin; tulokset tuonnin issueen.

Työvaihe 5 / 5 · Tarkista ja julkaise tuonti

☐ Kokeile rajoitteita. Kokeile omalla tiedostollasi, mitä svgelements ei lue. Kirjaa se project-docs/kirjastot.md:hen.

☐ Julkaise viikon versio. Tee tagi v0.0.43 ja pushaa se kuten viikolla 41. Kun Actions-ajo on vihreä, viikon versio on releasessa. Asiakkaat voivat kokeilla sitä.

Valmis kun: Kaksi itse kokeiltua rajoitetta on kirjattu ja viikon versio on julkaistu.

Tallenna työnäyte: Rajoitteet project-docs/kirjastot.md-tiedostoon; release v0.0.43 GitHubiin.

Valmis kun: Oma SVG avautuu tiedostoikkunasta, testit 3–5 menevät läpi, ja project-docs/kirjastot.md:ssä on kaksi svgelementsien rajoitetta omalla tiedostolla kokeiltuna.

Työnäyte Git-repositoryyn ennen rastia: oma SVG-tiedosto, testit 3–5 issueissa, project-docs/kirjastot.md ja release v0.0.43.

Vko 44 · 26.–30.10. – Piirroksen ryhmistä mallin osiksi

Piirroksen layerit ja ryhmät kertovat, mistä osista malli koostuu ja mitkä osat kuuluvat yhteen. Tällä viikolla säilytät tämän rakenteen Vektoripajassa, jotta käyttäjä voi käsitellä mallia osina. Sama rakenne tarvitaan myöhemmin osien siirtämiseen ja vientiin Blenderiin. Piirroksen tasot ja ryhmät näkyvät mallin nimettyinä osina.

Työvaihe 1 / 4 · Määrittele osien rakenteen testit

- ☐ Kirjoita rajapinta. Kirjoita korttiin funktion nimi, mitä se saa (SVG:n ryhmä) ja mitä se palauttaa (solmu, jolla on nimi, lapset ja oma muunnos). Rajapinta on funktion nimi, syöte ja paluuarvo.
- ☐ Kirjoita testit 6–8. Testi 6: kolme tasoa. Ryhmät ovat kolmessa tasossa. Testi 7: tyhjä ryhmä. Testi 8: nimetön ryhmä. Kirjaa jokaiselle, mitä odotat.

Valmis kun: Muunnoksen rajapinta ja testien 6–8 odotetut tulokset ovat kirjattuina.

Tallenna työnäyte: Rajapinta suunnitelmaan; testit tests-kansioon ja odotukset issueen.

Työvaihe 2 / 4 · Rakenna osien hierarkia

- ☐ Kirjoita maailmamuunnos. Maailmamuunnos kertoo osan lopullisen paikan näkymässä. Se lasketaan näin: vanhemman maailmamuunnos kertaa osan oma muunnos. Kirjoita siitä puhdas funktio ilman Qt:ta.
- ☐ Kirjoita ensin lisätesti: kun vanhempaa ei ole siirretty, kierretty eikä skaalattu, lapsen maailmamuunnos on sama kuin sen oma muunnos.
- ☐ Tee kortti Agentti-kaistalla. Kortti koskee funktiota ja paneelia eli useaa tiedostoa. Agentti-kaista tarkoittaa GitHub Copilotin agenttitilaa. Katso kuvaohjeet alta.

Valmis kun: Kolme ryhmätasoa näkyy sisennettynä ja hierarkian sekä maailmamuunnoksen testit menevät läpi.

Tallenna työnäyte: Koodi ja testit GitHubiin; tulokset toteutuksen issueen.

Työvaihe 3 / 4 · Päätä osan valinnan toiminta

- ☐ Päätä valinnan toiminta. Kun käyttäjä klikkaa osaa, valitaanko lapsi vai koko kappale?
- ☐ Kirjaa päätös ja peruste suunnitelmaan.

Valmis kun: Valinnan toiminta ja sen peruste on kirjattu.

Tallenna työnäyte: Valinnan päätös suunnitelmaan.

Työvaihe 4 / 4 · Valmistele seuraava mallinnustoiminto

- Vertaa revolveren valintatapoja. Vertaa kahta tapaa: valinta ja painike sovelluksessa, tai nimimerkintä Inkscapessa, esimerkiksi layerin nimen perässä [revolve]. Kirjaa hyvät ja huonot puolet. Tuo suositus ensi viikon palaveriin.
- Julkaise viikon versio. Tee tagi v0.0.44 ja pushaa se. Kun Actions-ajo on vihreä, viikon versio on releasessa.

Valmis kun: Kaksi valintatapaa on vertailtu ja suositus on valmis palaveriin.

Tallenna työnäyte: Vertailu suunnitelmaan; release v0.0.44 GitHubiin.

Valmis kun: Oman tiedostosi kolme ryhmätasoa näkyvät paneelissa sisennettyinä. Testit 6–8 ja maailmamuunnoksen lisätesti menevät läpi.

Suunnitelmassa on valinnan toiminta perusteluineen.

Työnäyte Git-repositoryyn ennen rastia: rajapinta, testit 6–8 ja maailmamuunnoksen lisätesti issueissa, hierarkiapaneeli releasessa v0.0.44 ja valinnan päätös suunnitelmassa.

Vko 45 · 2.–6.11. – Profiilista

pyörähdyskappale (revolve)

Nyt piirroksesta aletaan tehdä varsinaisia 3D-kappaleita. Revolve pyöräyttää profiilin akselin ympäri, jolloin siitä voi syntyä esimerkiksi maljakko tai pyörä.

Segmenttisäädin ja kameran kierto auttavat käyttäjää muotoilemaan ja tarkastelemaan tätä ensimmäistä mallinnustoimintoa.

Käyttäjä voi tehdä piirretystä puoliprofiilista pyörähdyskappaleen.

Työvaihe 1 / 3 · Sovi pyörähdyskappaleen toiminta

- ☐ Sovi valintatapa palaverissa. Esitä vertailusi. Sovi valintatapa ohjaajan kanssa. Kirjaa sovittu tapa issue-kommenttina ja suunnitelmaan.
- ☐ Päätä akselin väärän puolen käsittely. Mitä tehdään, jos profiilin piste on akselin väärällä puolella? Kirjaa oma hyväksymiskriteeri suunnitelmaan. Se on testin 11 odotettu tulos.

Valmis kun: Valintatapa ja akselin väärän puolen käsittely on sovittu.

Tallenna työnäyte: Päätökset suunnitelmaan ja hyväksymiskriteeri issueen.

Työvaihe 2 / 3 · Määrittele pyörähdyskappaleen testit

- ☐ Kirjoita testit 9–11. Testi 9: profiilin pisteet. Testi 10: segmenttien rajat, kun määrä on 2 ja 33. Testi

11: akselin väärä puoli. Profiilin piste on akselin väärällä puolella. Kirjaa odotukset ennen koodia.

□ Kysy kysymystilassa. Avaa GitHub Copilotin chat kysymystilassa eli Ask-tilassa. Kysy, miten `trimesh.creation.revolve` lukee pisteet. Kysymystila vastaa, mutta ei muuta tiedostoja.

Valmis kun: Testien 9–11 omat odotetut tulokset on kirjoitettu ennen toteutusta.

Tallenna työnäyte: Testit tests-kansioon ja odotukset issueen.

Työvaihe 3 / 3 · Rakenna ja julkaise pyörähdyskappale

□ Tee kortit työsyklillä. Tee `revolve`, segmenttisäädin ja `orbit`. `Orbit` tarkoittaa, että näkymää kierretään hiirellä. PyVistan kamerassa se on valmiina.

□ Julkaise viikon versio. Tee tagi `v0.0.45` ja pushaa se. Kun Actions-ajo on vihreä, viikon versio on releasessa.

Valmis kun: Oma profiili muuttuu kappaleeksi, segmenttimäärä muuttuu välillä 3–32 ja testit 9–11 menevät läpi.

Tallenna työnäyte: Koodi ja testit GitHubiin; release `v0.0.45` ja testitulokset issueen.

Valmis kun: Oma profiilisi pyörähtää kappaleeksi, segmenttisäädin muuttaa särmien määrää välillä 3–32, ja testit 9–11 menevät läpi.

Työnäyte Git-repositoryyn ennen rastia: testit 9–11 issueissa, pyörähdyskappale releasessa v0.0.45 ja sovittu valintatapa issue-kommenttina.

Vko 46 · 9.–13.11. – Viivasta putki ja mallin näkymät

Revolven rinnalle tarvitaan tapa tehdä piirroksen viivasta putkimainen osa. Tällä viikolla lisäät inflaten ja sen sivumäärän säädön sekä suorat kameranäkymät ja näkymän lukituksen. Näin Vektoripajalla voi rakentaa erilaisia osia ja tarkastella niitä vakaasta suunnasta.

Käyttäjä voi tehdä viivasta putken ja tarkastella mallia suorista näkymistä.

Työvaihe 1 / 3 · Määrittele putken ja kameran testit

☐ Tee ensin inflate. Inflate on pakollisen ytimen kohta 4 ja kamera kohta 6. Tee inflate ensin. Jos aika loppuu, kamera siirtyy viikon 47 alkuun. Siitä sovitaan palaverissa. Se on suunniteltu vaihtoehto.

☐ Kirjoita testit 12–14. Testi 12: sivujen rajat, kun määrä on 2 ja 9. Testi 13: suljettu polku. Testi 14: view lock. View lock on päällä, ja yrität kiertää näkymää. Kirjaa odotukset ennen koodia.

Valmis kun: Putken ja kameran hyväksymiskriteerit sekä testien 12–14 odotukset ovat kirjattuina.

Tallenna työnäyte: Testit tests-kansioon ja odotukset issueihin.

Työvaihe 2 / 3 · Rakenna viivasta putki

- ☐ Tee inflate työsyklillä. Tee putki PyVistan tube-suodattimella.
- ☐ Tee myös sivumäärän säädin. Kokeile omilla poluillasi. Kirjaa putkigeometrian rajoitteet project-docs/kirjastot.md:hen.

Valmis kun: Oma polku muuttuu putkeksi, sivumäärä muuttuu välillä 3–8 ja testit 12–13 menevät läpi.

Tallenna työnäyte: Koodi GitHubiin, testitulokset issueen ja rajoitteet kirjastot.md-tiedostoon.

Työvaihe 3 / 3 · Rakenna suorat näkymät

- ☐ Tee kamera työsyklillä. Orientaatiowidget on pieni akselikuvio näkymän kulmassa. Kun klikkaat sen akselia, kamera kääntyy katsomaan mallia suoraan akselin suunnasta. View lock on painike, joka estää kameran kiertymisen.
- ☐ Julkaise viikon versio. Tee tagi v0.0.46 ja pushaa se. Kun Actions-ajo on vihreä, viikon versio on releasessa.

Valmis kun: Akselin valinta kääntää näkymän suoraan, lukitus estää kierron ja testi 14 menee läpi, tai siirto viikolle 47 on sovittu.

Tallenna työnäyte: Kameran testitulos ja mahdollinen siirtopäätös issueen; release v0.0.46 GitHubiin.

Valmis kun: Inflate on valmis, kun oma polkusi muuttuu putkeksi, sivumäärän voi valita väliltä 3–8 ja testit 12–13 menevät läpi. Kamera on valmis, kun orientaatiowidgetin akselin klikkaus kääntää kameran suoraan akselin suuntaan, view lock estää kierron ja testi 14 menee läpi. Jos kamera siirtyy viikolle 47, se on sovittu vaihtoehto eikä virhe.

Työnäyte Git-repositoryyn ennen rastia: testit 12–14 issueissa, putki ja suorat näkymät releasessa v0.0.46 ja putkigeometrian rajoitteet kirjastot.md:ssä.

Vko 47 · 16.–20.11. – Osien siirto, kierto ja skaalaus

Kun osien muodot ovat valmiit, käyttäjän pitää voida koota niistä kokonainen malli. Tällä viikolla lisäät osien valinnan, siirron, kierron ja skaalauksen. Aiemmin rakennettu hierarkia varmistaa, että esimerkiksi vanhempaan kuuluva lapsiosa seuraa mukana.

Käyttäjä voi valita, siirtää, kiertää ja skaalata mallin osia.

Työvaihe 1 / 3 · Määrittele osien muokkaamisen testi

☐ Tuo pakollisen ytimen tilanne palaveriin.

Palaverissa katsotaan, onko pakollinen ydin aikataulussa. Jos ei ole, ohjaaja päättää, mikä katselmoidaan keskeneräisenä ja mikä tehdään viikolla 4. Jos kamera siirtyi tälle viikolle, tee se ensin.

☐ Kirjoita testi 15. Testi 15: lapsi seuraa.

Vanhempaa siirretään 10 yksikköä x-suunnassa. Mihin odotat lapsen siirtyvän? Kirjaa odotus ennen koodia.

Testi käyttää viikon 44 maailmamuunnosta.

Valmis kun: Testin 15 odotettu tulos on kirjoitettu ja pakollisen ytimen tilanne on sovittu palaverissa.

Tallenna työnäyte: Odotettu tulos ja palaverin päätös issueen; testi tests-kansioon.

Työvaihe 2 / 3 · Rakenna osien muokkaaminen

☐ Tee transformipaneeli työsyklillä.

Transformipaneelissa on numerokentät siirrolle, kierrolle ja skaalaukselle.

☐ Lisää pikanäppäimet suunnitelmasi mukaan. Käytä view lockia, kun siirrät osia.

Valmis kun: Valinta, siirto, kierto ja skaalaus toimivat sovitulla tavalla.

Tallenna työnäyte: Koodi GitHubiin ja muokkaamisen issueen linkki muutokseen.

Työvaihe 3 / 3 · Tarkista ja julkaise muokkaaminen

- ☐ Aja testi 15. Jos testi 15 ei mene läpi kahdella yrityksellä, avaa havaintoissue.
- ☐ Julkaise viikon versio. Tee tagi v0.0.47 ja pushaa se. Kun Actions-ajo on vihreä, viikon versio on releasessa.

Valmis kun: Testi 15 menee läpi, tai poikkeamasta on havaintoissue ja jatko on sovittu.

Tallenna työnäyte: Testitulokset tai havaintoissue GitHubiin; release v0.0.47.

Valmis kun: Osan voi valita suunnitelmasi mukaan. Siirto, kierto ja skaalaus toimivat numerokentillä ja pikanäppäimillä. Testi 15 menee läpi, tai siitä on havaintoissue.

Työnäyte Git-repositoryyn ennen rastia: testi 15 issuessa, transformit releasessa v0.0.47 ja pakollisen ytimen tilanne palaverin issue-komentissa.

Vaihe 3 – Tehdään ensimmäinen käyttökelpoinen versio

Vko 48 · 23.–27.11. – Kiertopisteen säätäminen (pivot)

Osan siirtäminen ja kiertäminen ei vielä riitä, jos liike tapahtuu väärän pisteen ympärillä. Tällä viikolla käyttäjä saa valita pivotin eli kierto- ja skaalauspisteen prosenttikentillä. Se täydentää osien muokkausta ja tekee esimerkiksi nivelen ympärillä kääntymisestä hallittavaa.

Käyttäjä voi säätää pistettä, jonka ympäri osa kiertää ja skaalautuu.

Työvaihe 1 / 3 · Määrittele kiertopisteen testit

- ☐ Kirjoita rajapinta. Pivotin syötteen tarkistus: nimi, syöte (prosentti) ja paluuarvo. Kirjaa korttiin.
- ☐ Kirjoita testit 16–18. Testi 16: pivot keskellä, 50/50/50. Testi 17: pivot reunoilla, 0 % ja 100 %. Testi 18: väärä pivot-syöte, -10, 150 ja teksti. Kirjaa, mitä odotat.
- ☐ Testin 18 odotus on oma kriteerisi: hylätäänkö arvo vai rajataanko se.

Valmis kun: Rajapinta ja testien 16–18 odotetut tulokset ovat valmiit ennen toteutusta.

Tallenna työnäyte: Rajapinta suunnitelmaan; testit ja odotukset issueen.

Työvaihe 2 / 3 · Rakenna kiertopisteen laskenta

- ☐ Tee pivot ja tarkistus työsyklillä. Tee ensin syötteen tarkistus puhtaana funktiona.
- ☐ Tee sitten pivotin siirto.

Valmis kun: Keskipiste ja prosenttisyöttö toimivat, virheellinen syöte käsitellään sovitusti ja testit 16–18 menevät läpi.

Tallenna työnäyte: Laskennan koodi ja testit GitHubiin; tulokset issueen.

Työvaihe 3 / 3 · Lisää kiertopisteen säätimet

- ☐ Lisää pivot transformipaneeliin. Lisää paneeliin kolme kenttää, joissa pivot syötetään prosentteina: X, Y ja Z. Kentillä on nimi, ja ne toimivat näppäimistöllä käyttöliittymävaatimuksesi mukaan.
- ☐ Julkaise viikon versio. Tee tagi v0.0.48 ja pushaa se. Kun Actions-ajo on vihreä, viikon versio on releasessa.

Valmis kun: Prosenttikentät muuttavat kiertopistettä sovelluksessa.

Tallenna työnäyte: Käyttöliittymän koodi ja release v0.0.48 GitHubiin.

Valmis kun: Pivot 50/50/50 on keskipisteessä, prosentit muuttavat pivotin paikkaa, virheellinen syöte käsitellään kriteerisi mukaan, ja testit 16–18 menevät läpi.

Työnäyte Git-repositoryyn ennen rastia: rajapinta ja testit 16–18 issueissa sekä pivotin kentät releasessa v0.0.48.

Vko 49 · 30.11.–4.12. – Mallin vienti Blenderiin (OBJ)

Vektoripajan mallin pitää olla käytettävissä myös sovelluksen ulkopuolella. Tällä viikolla viet sen OBJ-tiedostoksi ja varmistat Blenderissä, että nimetyt osat säilyvät erillisinä. Samalla dokumentoit virheen korjauksen ja lisäät testin, joka auttaa estämään saman virheen palaamisen.

Käyttäjä voi viedä mallin Blenderiin niin, että nimetyt osat säilyvät erillisinä.

Työvaihe 1 / 4 · Määrittele viennin testit

- ☐ Kirjoita rajapinta ja testit. Kirjoita objektijaon rajapinta korttiin.
- ☐ Testi 19: kolmen osan vienti. Testi 20: tyhjä vienti, kun mallissa ei ole osia. Kirjaa, mitä odotat.

Valmis kun: Objektijaon rajapinta ja testien 19–20 odotukset on määritelty.

Tallenna työnäyte: Rajapinta suunnitelmaan; testit tests-kansioon ja odotukset issueen.

Työvaihe 2 / 4 · Rakenna OBJ-vienti

- ☐ Tee vienti työsyklillä. Osien nimien pitää säilyä tiedostossa.
- ☐ Tarkista tulos ennen rastittamista. Jos tulos poikkeaa odotetusta, kirjaa havainto ja selvitä se ohjaajan kanssa.

Valmis kun: Viety tiedosto sisältää sovitut osat ja nimet.

Tallenna työnäyte: Viennin koodi ja esimerkin OBJ-tiedosto GitHubiin.

Työvaihe 3 / 4 · Kokeile mallia Blenderissä

- ☐ Tarkista tiedosto kahdella tavalla. Avaa .obj VS Codessa.
- ☐ Laske rivit, jotka alkavat o-kirjaimella. Avaa tiedosto sitten Blenderissä kuvaohjeen mukaan.

Valmis kun: Kolmen osan tiedostossa on kolme o-riviä ja malli avautuu Blenderissä.

Tallenna työnäyte: Testitulokset issueen ja kuva Blenderistä project-docs-kansioon.

Työvaihe 4 / 4 · Dokumentoi korjaus ja julkaise

- Kirjoita virheenkorkausketju 1. Valitse yksi havaintoissue. Kirjaa siihen kuusi osaa: havainto, toistamisohje, syy omin sanoin, korjauscommit, uusintatesti ja regressiotesti.
- Kirjoita regressiotesti. Regressiotesti on testi, joka toistaa korjatun virheen ja jää testeihin. Kirjoita se Täydennys-kaistalla kuten kohdassa 3a. Nimeä se issuen mukaan, esimerkiksi regressiotesti #12.
- Pyydä vikatehtävä, jos havaintoissueita ei ole. Kerro siitä palaverissa. Ohjaaja antaa vikatehtävän. Vikatehtävä on tarkoituksellinen virhe, josta ketju tehdään.
- Julkaise viikon versio. Tee tagi v0.0.49 ja pushaa se. Kun Actions-ajo on vihreä, viikon versio on releasessa.

Valmis kun: Ketjun kuusi osaa on kirjattu ja regressiotesti menee läpi.

Tallenna työnäyte: Korjausketju havaintoissueen, regressiotesti tests-kansioon ja release v0.0.49.

Valmis kun: Kolmen osan mallista syntyy .obj-tiedosto, jossa on kolme o-riviä osien nimillä. Tiedosto aukeaa Blenderissä. Testit 19–20 menevät läpi.

Virheenkorkausketjun kuusi osaa ovat havaintoissuessa.

Työnäyte Git-repositoryyn ennen rastia: testit 19–20 issueissa, viety .obj-tiedosto, kuva Blenderistä ja

virheenkorjausketju 1 havaintoissuessa.

Vko 50 · 7.–11.12. – Tallennus ja ensimmäinen toimiva versio

Käyttäjän pitää voida jättää mallinnustyö kesken ja jatkaa sitä myöhemmin samasta tilanteesta. Tallennus ja avaus täydentävät ensimmäisen toimivan version, jossa piirros kulkee tuonnista muokkaukseen ja vientiin.

Julkaiset tämän MVP:n versiona v0.1, jotta asiakkaat voivat kokeilla koko työnkulkua.

Käyttäjä voi tallentaa ja avata työnsä. Ensimmäinen toimiva versio v0.1 on ladattavissa.

Työvaihe 1 / 4 · Valitse tallennustapa

- ☐ Vertaa tallennustapoja. Käytä oman suunnitelmasi kriteerejä.
- ☐ Säilyvätkö polut, parametrit, transformit ja vanhempi–lapsi-suhteet? Vertaa ainakin JSON-tiedostoa ja Qt:n asetustallennusta QSettings. Kirjaa oman mallisi tallennustiedoston koko.

Valmis kun: Vaihtoehdot on vertailtu samoilla kriteereillä ja valinta on perusteltu.

Tallenna työnäyte: Vertailu, valinta ja oman tallennustiedoston koko suunnitelmaan.

Työvaihe 2 / 4 · Määrittele tallennuksen testit

- ☐ Kirjoita testit 21–22. Testi 21: tallennus ja avaus.
- ☐ Tallennat mallin ja avaat sen. Testi 22: rikottu tallennus. Avaat valitun tallennusmuodon rikotun tiedoston. Kirjaa, mitä odotat.

Valmis kun: Testien 21–22 syötteet ja omat odotetut tulokset on kirjattu.

Tallenna työnäyte: Testit tests-kansioon ja odotukset issueen.

Työvaihe 3 / 4 · Rakenna tallennus ja avaus

- ☐ Tee tallennus ja avaus työsyklillä. Tallenna tiedostoon versionumero. Silloin vanhan tiedoston tunnistaa myöhemmin.
- ☐ Kirjoita tietoturva-arvio. Tee taulukko: uhka, testi, tulos ja toimenpide. Käy läpi ainakin haitallinen SVG (testi 4) ja rikottu tallennus (testi 22). Tallenna arvio tiedostoon `project-docs/tietoturva.md`.

Valmis kun: Tallennettu työ avautuu samassa tilassa ja rikottu tiedosto antaa hallitun virheilmoituksen.

Tallenna työnäyte: Koodi ja testitulokset GitHubiin; tietoturva-arvio `project-docs/tietoturva.md`-tiedostoon.

Työvaihe 4 / 4 · Julkaise ensimmäinen toimiva versio

- ☐ Julkaise MVP. Tee tagi `v0.1` ja pushaa se.

- Kun Actions-ajo on vihreä, lataa zip releasesta. Pura se ja käynnistä sovellus puretusta kansiota.

Valmis kun: Releasen v0.1 zipistä purettu sovellus käynnistyy.

Tallenna työnäyte: Release v0.1 GitHubiin ja käynnistyskokeilun tulos issueen.

Valmis kun: Tallennettu projekti avautuu samassa tilassa, rikottu tiedosto antaa virheilmoituksen, project-docs/tietoturva.md on repositoryssa, ja releasen v0.1 zipistä purettu sovellus käynnistyy.

Työnäyte Git-repositoryyn ennen rastia:

tallennusvertailu suunnitelmassa, testit 21–22 issueissa, project-docs/tietoturva.md ja release v0.1.

Vko 51 · 14.–17.12. – Asiakkaat kokeilevat ensimmäistä versiota

MVP on nyt valmis asiakkaiden kokeiltavaksi, joten tarkistat heidän kanssaan, vastaako se sovittua tarvetta. Katselmoinnin havainnot ja asiakkaiden prioriteetit ohjaavat korjauksia sekä joululoman jälkeistä jatkokehitystä. Tilatiedostoon kirjattu seuraava tehtävä auttaa jatkamaan samasta kohdasta tauon jälkeen.

Asiakkaat ovat kokeilleet ensimmäistä versiota ja seuraavat muutokset on sovittu.

Työvaihe 1 / 4 · Valmistele asiakkaiden kokeilu

- ☐ Varaa katselmointiaika. Varaa aika asiakkaiden kanssa Teamsissa. Lähetä Antille releasen v0.1 osoite etukäteen, jotta hän ehtii ladata zipin omalle Windows-koneelleen. Tarkista ohjaajalta, miten katselmointi kirjataan.
- ☐ Valmistele demo. Näytä viidessä minuutissa omalla Inkscape-tiedostollasi: tuonti, revolve, inflate, transformit ja vienti. Harjoittele kerran ajastettuna.

Valmis kun: Aika on sovittu, oma SVG ja demo ovat valmiit ja Antilla on latauslinkki.

Tallenna työnäyte: Demon rakenne ja releasen osoite katselmointimuistioon.

Työvaihe 2 / 4 · Kerää asiakkaiden palaute

- ☐ Pidä katselmointi. Anna asiakkaiden kokeilla itse. Antti kokeilee releasen zipiä omalla koneellaan. Matti kokeilee sovellusta sinun koneellasi. Kirjaa havainnot lokiin omin sanoin tiivistettynä. Älä kirjoita julkiseen repositoryyn nimiä tai sanatarkkoja lausumia.
- ☐ Kirjoita tulkinta erikseen. Kirjoita oma tulkintasi lokin kohtaan Oma tulkinta. Älä sekoita sitä asiakkaiden havaintoihin.

Valmis kun: Molemmat asiakkaat ovat kokeilleet versiota ja havainnot on erotettu omasta tulkinnasta.

Tallenna työnäyte: Roolit ja tulkinta project-docs/katselmointi.md-tiedostoon; nimet ja sanatarkat lausumat Teamsiin.

Työvaihe 3 / 4 · Muuta havainnot tehtäviksi

- ☐ Tee havainnoista issuet. Tee jokaisesta havainnosta issue.
- ☐ Kirjaa siihen asiakkaan antama prioriteetti.

Valmis kun: Jokaisesta havainnosta on issue ja asiakkaan prioriteetti.

Tallenna työnäyte: Havainnot GitHub-issueihin.

Työvaihe 4 / 4 · Kirjaa loman jälkeinen jatko

- ☐ Päivitä tilatiedosto. Kirjoita Seuraavana-kohtaan viikon 2 ensimmäinen tehtävä ja sen issue-numero.
- ☐ Tee commit ennen lomaa.

Valmis kun: Seuraavana-kohdassa on viikon 2 ensimmäinen tehtävä ja issue-numero.

Tallenna työnäyte: Päivitetty PROJEKTIN-TILA.md repositoryn juureen.

Valmis kun: Katselmointiloki on tiedostossa project-docs/katselmointi.md, jokaisesta havainnosta on issue prioriteetteineen, ja tilatiedoston Seuraavana-kohdassa on viikon 2 ensimmäinen tehtävä issue-numeroineen.

Työnäyte Git-repositoryyn ennen rastia: project-docs/katselmointi.md, havaintoissuet prioriteetteineen ja tilatiedoston Seuraavana-kohta.

Vko 52 · 21.–25.12. – Joululoma

Joululoma alkaa pe 18.12. Ei projektityötä eikä korvaavia tehtäviä.

Vko 53 · 28.12.–1.1. – Joululoma

Joululoma jatkuu. Ei projektityötä eikä korvaavia tehtäviä.

Vko 1 · 4.–8.1. – Joululoma

Joululoma päättyy su 10.1. Jatka ma 11.1. viikolla 2 tilatiedoston Seuraavana-kohdasta.

Vaihe 4 – Parannetaan asiakkaiden palautteen perusteella

Vko 2 · 11.–15.1. – Paluu ja parannusten järjestys

Joululoman jälkeen varmistat ensin, että kehitysympäristö, sovellus ja aiemmat testit toimivat edelleen. Sen jälkeen sovit tärkeän jatkon järjestyksen asiakkaiden katselmointipalautteen perusteella ja teet yhden pienen korjauksen. Näin uudet ominaisuudet rakentuvat toimivan MVP:n ja todellisten käyttäjätarpeiden päälle.

Aiempi versio toimii loman jälkeen ja parannusten järjestys on sovittu.

Työvaihe 1 / 3 · Varmista aiemman version toiminta

- ☐ Käynnistä ympäristö. Avaa projekti VS Codessa. Avaa uusi terminaali. Tarkista, että terminaalin rivin alussa lukee (.venv). Aja python tarkista_ymparisto.py, sitten python main.py ja lopuksi pytest. Jos jokin ei toimi, katso työsyklin kohta Olen jumissa.
- ☐ Kirjaa tulos päiväkirjaan.
- ☐ Lue tilatiedosto. Avaa PROJEKTIN-TILA.md. Aloita uusi Copilot-keskustelu tilatiedostolla.

Valmis kun: Ympäristötarkistus ja kaikki testit menevät läpi; tilatiedosto on luettu.

Tallenna työnäyte: Ajotulokset viikon päiväkirjaan.

Työvaihe 2 / 3 · Sovi parannusten järjestys

- ☐ Ehdota tärkeän jatkon järjestys. Asiakkaat antoivat prioriteetit katselmoinnissa. Ehdota järjestys. Arvioi jokaiselle toiminnolle tunnit.
- ☐ Arvioi MVP palaverissa. Käy palaverissa läpi, mikä MVP:n ratkaisu kestää tärkeän jatkon toiminnot ja mikä ei. Ohjaaja on tässä tiimin jäsenen roolissa. Sovi järjestys.

Valmis kun: Parannusten järjestys ja tuntiarviot on sovittu palaverissa.

Tallenna työnäyte: Järjestys ja tuntiarviot suunnitelmaan sekä sovitut tehtävät issueihin.

Työvaihe 3 / 3 · Tee ensimmäinen korjaus

- ☐ Tee pieni korjaus Täydennys-kaistalla. Valitse katselmoinnin havaintoissueista pieni korjaus. Tee se työsyklillä.
- ☐ Tarkista krediitit. Budjetti nollautui 1.1. Kirjaa tammikuun lähtötilanne päiväkirjaan.

Valmis kun: Sovittu korjaus on testattu ja GitHubissa.

Tallenna työnäyte: Korjauscommit ja testitulos issueen; krediittitilanne päiväkirjaan.

Valmis kun: python tarkista_ymparisto.py tulostaa joka rivin alkuun OK, pytest näyttää, että kaikki testit menevät läpi, tärkeän jatkon järjestys on sovittu palaverissa, ja korjauksen commit on GitHubissa.

Työnäyte Git-repositoryyn ennen rastia: testiajon tulos päiväkirjassa, tärkeän jatkon järjestys issueissa ja korjauksen commit.

Vko 3 · 18.–22.1. – Muokatun piirroksen päivittäminen

Inkscape-piirros voi muuttua myös sen jälkeen, kun siitä on tehty malli. Nyt lisäät Päivitä SVG -toiminnon ja sovit, miten aiemmat osat tunnistetaan uudesta piirroksesta. Liität toiminnon testattuna aiempaan sovellukseen oman haaran ja pull requestin avulla.

Käyttäjä voi päivittää muutetun SVG-piirroksen malliin ja säilyttää osien muokkaukset.

Työvaihe 1 / 3 · Sovi osien tunnistaminen

- ☐ Sovi tunnistustapa. Sovi palaverissa, miten osa tunnistetaan uudesta tiedostosta: nimellä vai tunnisteella.
- ☐ Kirjaa päätös ja peruste suunnitelmaan.

Valmis kun: Osien tunnistustapa on sovittu ja perusteltu.

Tallenna työnäyte: Päätös suunnitelmaan.

Työvaihe 2 / 3 · Määrittele päivityksen testi

- ☐ Kirjoita testi 23. Testi 23: Päivitä SVG. Muokkaa testitiedostoa Inkscapeissa ja määrittele, mitä odotat mallin osien muokkauksille päivityksen jälkeen.
- ☐ Kirjoita oma odotettu tulos ennen toteutusta. Kirjoita sitten testi, joka tarkistaa tämän tuloksen.

Valmis kun: Testin 23 odotettu tulos on kirjattu ennen toteutusta.

Tallenna työnäyte: Testi tests-kansioon ja odotukset issueen.

Työvaihe 3 / 3 · Rakenna piirroksen päivitys

- ☐ Tee oma haara. Haara on oma työlinja. Muutokset eivät vaikuta päähaaraan main, ennen kuin liität ne.
- ☐ Tee toiminto työsyklillä haarassa. Tee kortit tavalliseen tapaan. Tee commitit haaraan.
- ☐ Aja kaikki testit. Aja pytest. Kaikkien vanhojen testien pitää mennä läpi.
- ☐ Tee pull request ja merge. Pull request on pyyntö liittää haara päähaaraan. Lue muutokset itse. Tee sitten merge eli liitä haara päähaaraan.

Valmis kun: SVG päivittyy, osien muokkaukset säilyvät, kaikki testit menevät läpi ja pull request on yhdistetty.

Tallenna työnäyte: Koodi, testi ja yhdistetty pull request GitHubiin.

Valmis kun: Muokattu SVG päivittyy malliin, transformit säilyvät, kaikki testit menevät läpi, ja pull request on yhdistetty päähaaraan.

Työnäyte Git-repositoryyn ennen rastia: testi 23 issuessa ja yhdistetty pull request.

Vko 4 · 25.–29.1. – Puutteet ja sovitut parannukset

Tämä viikko antaa tilaa varmistaa, että Vektoripajan pakollinen ydin on valmis ennen seuraavia laajennuksia. Korjaat ensin sovitut rästit ja etenet sen jälkeen asiakkaiden kanssa päätetyssä tärkeän jatkon järjestyksessä. Uusi testi ja toinen virheenkorjausketju auttavat pitämään myös jatkokehityksen laadun näkyvänä.

Sovittu puute on korjattu tai seuraava tärkeä jatkotoiminto on valmis.

Työvaihe 1 / 3 · Tarkista perustoimintojen puutteet

- ☐ Tarkista rästit. Katso palaverissa, jäikö pakollisesta ytimestä jotain kesken.
- ☐ Jos jäi, tee se ensin omassa haarassa. Liitä se päähaaraan pull requestilla.

Valmis kun: Rästit on tunnistettu ja sovittu, tehdäänkö tällä viikolla korjaus vai jatkotoiminto.

Tallenna työnäyte: Tilanne ja sovittu tehtävä issueen sekä tilatiedostoon.

Työvaihe 2 / 3 · Määrittele seuraavan toiminnon testi

- ☐ Valitse tärkeän jatkon toiminto. Jos rästejä ei ole, ota järjestyksen seuraava toiminto.
- ☐ Kirjoita testi 24. Testi 24: seuraava toiminto. Kirjoita toiminnon testi ennen toteutusta. Esimerkiksi kulmasnappauksessa: mitä odotat, kun kulma on 22° ja kun se on 23°?

Valmis kun: Testin 24 syöte ja odotettu tulos on kirjoitettu ennen toteutusta.

Tallenna työnäyte: Valinta suunnitelmaan ja testi sekä odotukset issueen.

Työvaihe 3 / 3 · Toteuta sovittu parannus

- ☐ Tee toiminto työsyklillä. Käytä haaraa, jos muutos koskee useaa tiedostoa.
- ☐ Kirjoita virheenkorjausketju 2. Valitse havaintoissue ja kirjaa kuusi osaa kuten viikolla 49. Kirjoita regressiotesti Täydennys-kaistalla.

Valmis kun: Sovittu muutos toimii, testit menevät läpi ja virheenkorjausketju 2 on kirjattu.

Tallenna työnäyte: Muutos GitHubiin pull requestilla; ketju havaintoissueen.

Valmis kun: Pakollisen ytimen rästit on liitetty päähaaraan pull requestilla, tai tärkeän jatkon toiminto toimii ja testi 24 menee läpi. Virheenkorjausketjun kuusi osaa ovat havaintoissuessa.

Työnäyte Git-repositoryyn ennen rastia: testi 24 issuessa, valmis toiminto päähaarassa ja virheenkorjausketju 2 havaintoissuessa.

Vko 5 · 1.–5.2. – Selkeät säätimet ja näppäimistökäyttö

Valmiista mallinnustoiminnoista on hyötyä vain, jos käyttäjä pystyy käyttämään niiden säätimiä. Tällä viikolla parannat isoja säätimiä ja näppäimistökäyttöä sekä kokeilet sovellusta Lukijalla. Ennen ja jälkeen tehdyt tarkistukset näyttävät, miten muutokset parantavat Vektoripajan käytettävyyttä.

Sovelluksen säätimiä voi käyttää selkeämmin näppäimistöllä ja Lukijalla.

Työvaihe 1 / 4 · Mittaa saavutettavuus

- ☐ Mittaa ensin. Käynnistä sovellus.
- ☐ Avaa Accessibility Insights for Windows. Valitse FastPass. Valitse kohteeksi Vektoripajan ikkuna. Aja

tarkistus kuvaohjeen mukaan. Kirjaa virheiden määrä ja kolme ensimmäistä havaintoa päiväkirjaan.

Valmis kun: FastPassin tulokset on kirjattu ennen muutoksia.

Tallenna työnäyte: Mittaus project-docs-kansioon ja havaintojen linkit issueihin.

Työvaihe 2 / 4 · Määrittele käyttöliittymän testit

- ☐ Tarkista vaatimus. Lue käyttöliittymävaatimuksesi suunnitelmasta. Mitä vielä puuttuu?
- ☐ Kirjoita testit 25–26. Testi 25: näppäimistö. Käytä kaikkia painikkeita näppäimistöllä. Testi 26: painikkeiden nimet. Käynnistä Lukija näppäimillä Ctrl + Windows-näppäin + Enter. Siirry Tab-näppäimellä painikkeesta toiseen. Lukeeko Lukija jokaisen painikkeen nimen? Kirjaa, mitä odotat.

Valmis kun: Oma vaatimus ja testien 25–26 odotetut tulokset on kirjattu.

Tallenna työnäyte: Vaatimus suunnitelmaan ja testit issueihin.

Työvaihe 3 / 4 · Paranna säätimien käyttöä

- ☐ Tee isot säätimet ja näppäimistö työsyklillä. Suurennä painikkeet ja transformipaneelin kentät.
- ☐ Anna jokaiselle painikkeelle nimi, jonka Lukija lukee. Tarkista, että Tab-järjestys on looginen. Tee

raahattavat kahvat vain, jos ne kuuluvat tärkeän jatkon järjestykseen.

Valmis kun: Sovitut säätimet ja näppäimistökäyttö on toteutettu.

Tallenna työnäyte: Koodi ja muutosten linkit GitHub-issueihin.

Työvaihe 4 / 4 · Tarkista muutosten vaikutus

- ☐ Mittaa uudelleen. Aja testit 25–26 ja FastPass uudelleen.
- ☐ Vertaa tuloksia. Kirjaa, mikä parani ja mikä jäi.

Valmis kun: Testit on kokeiltu näppäimistöllä ja Lukijalla; uudet havainnot on korjattu tai kirjattu.

Tallenna työnäyte: Ennen–jälkeen-vertailu project-docs-kansioon ja testitulokset issueihin.

Valmis kun: FastPassin tulokset ennen ja jälkeen on kirjattu, jälkimittauksessa ei ole uusia virheitä, testit 25–26 menevät läpi näppäimistöllä ja Lukijalla, ja löydetyt puutteet on korjattu tai kirjattu issueiksi.

Työnäyte Git-repositoryyn ennen rastia: FastPassin tulokset ennen ja jälkeen, testit 25–26 issueissa ja korjausten commitit.

Vaihe 5 – Julkaisu ja näyttö

Vko 6 · 8.–12.2. – Julkaisun kokeilu toisella koneella

Vektoripajan pitää toimia myös ihmiselle, joka ei tunne sen kehitystä tai omista Python-kehitysympäristöä. Tällä viikolla julkaisutestaaja kulkee ohjeen avulla koko polun latauksesta omaan piirrokseen ja OBJ-vientiin.

Julkaisuehdokkaan kokeilu paljastaa viimeiset esteet sovelluksessa ja käyttöohjeessa ennen v1.0:aa.

Toinen käyttäjä pystyy kulkemaan ohjeen avulla latauksesta omaan malliin ja vientiin.

Työvaihe 1 / 3 · Valmistele julkaisuehdokas

- ☐ Varaa testiaika ja kone. Sovi julkaisutestaajan kanssa aika. Varmista, että hänen Windows-koneellaan on Inkscape. Varmista myös, ettei koneelle ole asennettu Pythonia. Silloin näet, toimiiko zip ilman Pythonia.
- ☐ Jäädytä ja tee tagi. Jäädytä sisältö. Tee tagi v1.0-rc1 ja pushaa se. Kun Actions-ajo on vihreä, release ja sen zip ovat GitHubissa.

Valmis kun: Testiaika ja Windows-kone on sovittu, sisältö jäädytetty ja tagi v1.0-rc1 tehty.

Tallenna työnäyte: Tagi GitHubiin ja testijärjestelyt julkaisutestimuistioon.

Työvaihe 2 / 3 · Kirjoita käyttäjän ohje

- ☐ Täydennä README. Kirjoita README ja käyttöohje.
- ☐ Kerro, mistä zip ladataan, miten se puretaan ja mitä SmartScreen-varoituksessa valitaan. Kerro Inkscapen nimeämissäännöt: layerit, ryhmät ja mahdolliset nimimerkinnyt.

Valmis kun: Ohje kattaa lataamisen, piirroksen nimeämisen, mallintamisen ja viennin.

Tallenna työnäyte: README ja käyttöohje GitHubiin.

Työvaihe 3 / 3 · Kokeile julkaisua toisella koneella

- ☐ Pidä julkaisutesti. Anna testaajalle vain README:n osoite. Seuraa, älä neuvo. Testaaja lataa zipin releasesta ja purkaa sen. Sitten hän kulkee polun: piirros, tuonti, revolve tai inflate ja vienti.
- ☐ Listaa epäröinnit. Kirjaa pöytäkirjaan jokainen kohta, jossa testaaja epäröi. Kirjoita se omin sanoin ja roolilla "julkaisutestaaja". Lähetä testaajan nimi ja sanatarkat lausumat ohjaajalle Teamsissa.
- ☐ Korjaa ohje. Korjaa README jokaisen epäröinnin kohdalta. Tee estävistä virheistä issuet viikolle 7.

Valmis kun: Testaajan OBJ avautuu ja kaikki epäröinnit on kirjattu; ohjeen puutteet on korjattu.

Tallenna työnäyte: Roolit ja havainnot project-docs/julkaisutesti.md-tiedostoon; nimet ja sanatarkat lausumat Teamsiin.

Valmis kun: Julkaisutestaajan .obj-tiedosto aukeaa, ja epäröinnit on listattu pöytäkirjaan. Testaajan nimi ja sanatarkat lausumat on lähetetty ohjaajalle Teamsissa. Repositoryyn on kirjattu vain rooli.

Työnäyte Git-repositoryyn ennen rastia: tagi v1.0-rc1, README ja käyttöohje sekä project-docs/julkaisutesti.md.

Vko 7 · 15.–19.2. – Valmis sovellus v1.0

Julkaisutestissä löytyneet esteet korjataan, jotta Vektoripaja voidaan luovuttaa asiakkaiden käyttöön. Julkaiset version v1.0 ja pyydät asiakkaita vahvistamaan sovelluksen toimivuuden. Samalla kokoat työnäytteiden linkit, jotta projektin ratkaisut ja oma osaamisesi löytyvät näyttöä varten.

Vektoripaja v1.0 on julkaistu ja asiakkaat ovat vahvistaneet toimivuuden.

Työvaihe 1 / 3 · Korjaa julkaisun esteet

- ☐ Korjaa estävät havainnot. Korjaa viikon 6 estävät issuet työsyklillä.

- ☐ Kirjoita virheenkorjausketju 3. Valitse yksi korjauksista. Kirjaa ketjun kuusi osaa kuten viikolla 49.

Valmis kun: Estävät havainnot on korjattu ja virheenkorjausketju 3 on kirjattu.

Tallenna työnäyte: Korjauscommitit, uusintatestit ja ketju havaintoissueihin.

Työvaihe 2 / 3 · Julkaise valmis sovellus

- ☐ Julkaise v1.0. Tee tagi v1.0 ja pushaa se. Kun Actions-ajo on vihreä, kirjoita release-tekstiin, mitä versiossa on.
- ☐ Pyydä asiakkaiden vahvistus. Lähetä asiakkaille releasen osoite viestipohjalla. Antti lataa zipin. Hän kokeilee sovellusta omalla Windows-koneellaan. Matilla on Mac, joten näytä sovellus hänelle palaverissa omalla koneellasi tai demossa. Mac-versiota ei tehdä.

Valmis kun: Release v1.0 on ladattavissa ja asiakkaat ovat vahvistaneet toimivuuden.

Tallenna työnäyte: Release GitHubiin ja vahvistukset rooleilla projektin aineistoon.

Työvaihe 3 / 3 · Kokoa työnäytteiden linkit

- ☐ Linkitä näyttömatriisi. Työnäyte on yksi tuotos, joka osoittaa osaamisesi.

□ Päiväkirjassa on jokaisen viikon kohdalla näyttömatriisin vaatimukset. Kopioi päiväkirjasta linkit näyttömatriisin oikeille riveille.

Valmis kun: Jokaisella näyttömatriisin rivillä on täsmällinen linkki.

Tallenna työnäyte: Linkit näyttömatriisiin ja koko päiväkirja GitHubiin.

Valmis kun: Estävät issuet on suljettu, release v1.0 ja sen zip näkyvät GitHubissa, Antti on vahvistanut toimivuuden omalla Windows-koneellaan, Matti on nähnyt sovelluksen demossa tai sinun koneellasi, ja jokaisella matriisin rivillä on linkki.

Työnäyte Git-repositoryyn ennen rastia: tagi v1.0, virheenkorjausketju 3, asiakkaiden vahvistus ja linkitetty näyttömatriisi.

Vko 8 · 22.–26.2. – Talviloma

Ei projektityötä eikä korvaavia tehtäviä. Jatka viikolla 9 näytön valmistelulla.

Vko 9 · 1.–5.3. – Projektin esittely ja luovutus

Projektin päätöksessä näytät sekä valmiin Vektoripajan että sen, miten rakensit sen. Demo yhdistää käyttäjän työnkulun, yhden teknisen ratkaisun, virheenkorjauksen

ja versionhallinnan konkreettiseksi kokonaisuudeksi. Itsearviointi ja linkitetty aineisto täydentävät asiakkaalle ja arvioijalle tehtävän luovutuksen.

Valmis sovellus, työskentelysi ja oma osaamisesi on esitelty ja aineisto luovutettu.

Työvaihe 1 / 3 · Tarkista luovutettava aineisto

- ☐ Jäädytä sisältö maanantaina. Älä lisää enää ominaisuuksia.
- ☐ Tarkista, että jokainen työnäyte on repositoryssa.

Valmis kun: Sisältö on jäädytetty ja työnäytteet ovat repositoryssa.

Tallenna työnäyte: Repositoryn osoite ja aineiston linkit päiväkirjaan.

Työvaihe 2 / 3 · Harjoittele projektin esittely

- ☐ Harjoittele demo ajastettuna. Näytä viisi osaa: tuotos, yksi tekninen ratkaisu omin sanoin, korjattu virhe, Git-historia ja työnkulku yhdellä kortilla.
- ☐ Näytä työnkulku yhdellä kortilla. Näytä yhden kortin polku: oma rajaus ja odotettu tulos, issue, GitHub Copilot, perusteltu hylkäys tai korjautus ja commit.

Valmis kun: Ajastettu 8–10 minuutin demo sisältää sovitut viisi osaa.

Tallenna työnäyte: Demon runko ja harjoittelun havainto päiväkirjaan.

Työvaihe 3 / 3 · Arvioi ja luovuta työsi

- ☐ Kirjoita itsearviointi. Valitse päiväkirjasta kolme tilannetta. Niistä on kirjattu päivä ja issue-numero. Kirjoita arvio alla olevaan pohjaan. Lähetä se ohjaajalle Teamsissa, ei repositoryyn.
- ☐ Luovuta aineisto. Luovuta näyttöaineisto perjantaina 5.3.2027.

Valmis kun: Itsearviointi on lähetetty ohjaajalle ja sovellus sekä aineisto on esitelty ja luovutettu.

Tallenna työnäyte: Itsearviointi Teamsiin; lähetys- ja luovutuspäivä päiväkirjaan.

Valmis kun: Demo on pidetty 8–10 minuutissa, kaikki viisi osaa on näytetty, itsearviointi on lähetetty ohjaajalle, ja aineisto on luovutettu perjantaina.

Työnäyte Git-repositoryyn ennen rastia: demo, linkitetty näyttömatriisi ja itsearviointin lähetyspäivä päiväkirjassa.

Viimeiset viisi päivää

Ma 1.3. · Sisältöjääditys: työnäytteet repositoryssa

Ti 2.3. · Aineisto: päiväkirja, AI-loki ja näyttömatriisin linkit

Ke 3.3. · Demon harjoittelu ajastettuna toiselle henkilölle

To 4.3. · Puskuri ja itsearviointi Teamsissa ohjaajalle

Pe 5.3. · Demo ja luovutus

Sanasto

Projektin tunnukset ja ammattitermit siinä järjestyksessä, jossa ne tulevat vastaan. Jokainen on selitetty myös sivustolla ensimmäisen käytön kohdalla.

Copilot vko 40	Microsoft 365 Copilot. Tekoäly selaimessa BC:n tunnuksella. Suunnittelee ja pilkkoo viikon tavoitteen tehtäväkorteiksi. Tässä projektissa sana Copilot tarkoittaa aina tätä.
GitHub Copilot vko 40	koodityökalu VS Codessa. Tekoäly, joka toteuttaa tehtäväkortin VS Codessa. Tässä projektissa sana GitHub Copilot tarkoittaa aina tätä.
MVP vko 40	Minimum Viable Product. Pienin käytettävä versio. Tässä projektissa MVP on sama kuin pakollinen ydin ja valmistuu ennen joulua.
pakollinen ydin vko 40	Toiminnot, joiden pitää valmistua ennen joulua. Ilman niitä tuotosta ei ole.

tärkeä jatko vko 40	Toiminnot, jotka tehdään joululoman jälkeen, kun pakollinen ydin toimii.
jatkolista vko 40	Toiminnot, jotka jäävät tämän näytön jälkeen tehtäviksi. Ne eivät ole hylättyjä.
SVG vko 40	vektorikuva. Kuvan tiedostomuoto, jossa kuva koostuu poluista. Inkscape tallentaa piirrokset SVG-tiedostoiksi.
low-poly vko 40	3D-malli, jossa on vähän monikulmioita. Pinnat näkyvät tasaisina, kulmikkaina paloina.
tulkki vko 40	Python-tulkki. Ohjelma, joka ajaa Python-koodin. Komento python käynnistää tulkin. Tässä projektissa käytetään Python 3.13:a.

pip vko 40	Pythonin paketinhallinta. Työkalu, joka asentaa Python-kirjastoja. Se tulee Pythonin mukana. Kirjastot asennetaan komennolla <code>pip install</code> virtuaaliympäristöön.
virtuaaliympäristö vko 40	.venv. Projektin oma Python-kansio <code>.venv</code> . Projektin kirjastot asennetaan sinne, joten ne eivät sekoitu koneen muihin Python-projekteihin. Kun virtuaaliympäristö on päällä, terminaalin rivin alussa lukee <code>(.venv)</code> . Se luodaan viikolla 41.
repository vko 40	Git-repository. Projektin kansio GitHubissa. Siellä on koodi, historia ja issuet. Kaikki työnäytteet ovat repositoryssa.
README vko 40	Repositoryn etusivun ohje. Kertoo, mikä työkalu on ja miten sen saa käyttöön.

kloonaus vko 40	Repositoryn kopiointi omalle koneelle. VS Code ja GitHub Copilot käsittelevät kloonattua kansiota.
commit vko 40	Yksi nimetty muutos Gitissä. Commitilla on viesti ja tunnus.
push vko 40	Commitien lähetys omalta koneelta GitHubiin.
Collaborator vko 40	Henkilö, jolle repository on jaettu. Ohjaaja näkee ja voi kommentoida repositoryä.
tilatiedosto vko 40	PROJEKTIN-TILA.md. Tiedosto, jossa on kohdat Valmista, Seuraavana, Tehdyt päätökset ja Avoimet kysymykset. Liität sen Copilotiin. Älä anna sitä GitHub Copilotille.
viestipohja vko 40	Valmis viesti, jonka kopioit ja täydennät. Sivun Kopioi-nappi kopioi sen.

krediitti vko 40	GitHub Copilotin käyttöraha. 200 krediittiä kuukaudessa. Yksi krediitti on 0,01 USD. Agenttitila kuluttaa niitä. Täydennykset ovat ilmaisia. Budjetti nollautuu kuun 1. päivänä, eikä käyttämätön osuus siirry.
agenttitila vko 40	Agent. GitHub Copilotin tila, joka muokkaa useaa tiedostoa. Kuluttaa krediittejä.
käyttönäkymä vko 40	GitHubin sivu, jolla krediittien kulutus näkyy.
projektipäiväkirja vko 40	Tämän sivun viikkokirjaus: mitä teit, miksi ja missä työnäyte on. Ladataan kansioon project-docs joka viikko.
AI-loki vko 40	Tämän sivun loki tekoälyn käytöstä: mihin pyysit apua, hyväksyitkö, korjautitko vai hylkäsitkö ja miksi.
viikkopalaveri vko 40	Viikon tapaaminen ohjaajan kanssa maanantaina tai tiistaina. Tiimi tarkoittaa tässä projektissa sinua ja ohjaajaa.

työsykli vko 41	kuusi askelta. Suunnittele, Siirrä, Rakenna, Tarkista, Raportoi ja Kirjaa. Sama jokaisessa tehtäväkortissa.
tehtäväkortti vko 41	Yksi rajattu tehtävä: tavoite, kaista, tiedostot, Älä tee -kohta, hyväksymiskriteerit ja testi.
hyväksymiskriteeri vko 41	Havaittava ehto, jonka pitää toteutua, jotta kortti on valmis. Esimerkiksi: kuutio näkyy ja pyörii.
issue vko 41	GitHub-issue. Tehtäväkortti GitHubissa, label tehtäväkortti. Suljetaan commit-viestin rivillä Closes #N.
mallikortti vko 41	Valmis esimerkkikortti, johon omaa korttia verrataan. Viikolla 41 siinä on myös testipohja.

kaista vko 41	Tiedosto, Täydennys tai Agentti. Toteutustapa. Tiedosto-kaista: Copilot kirjoittaa koodin yksi tiedosto kerrallaan. Täydennys-kaista (viikolta 43): kommentti ja täydennys VS Codessa, myös testit. Agentti-kaista (viikolta 44): GitHub Copilotin agenttitila useaan tiedostoon.
testi vko 41	numeroitu testitapaus. Testillä on numero ja nimi, esimerkiksi testi 1: kiertokulma. Numero kertoo järjestyksen: testi 1 on ensimmäinen, testi 2 toinen. Odotettu tulos kirjataan, ennen kuin koodi tehdään. Lisätesti on ylimääräinen testi ilman numeroa.

tekeminen pohja vko 41	PySide6, PyVista, trimesh ja pytest. Sovelluksen perusta. PySide6 (Qt) tekee ikkunan ja painikkeet. PyVista piirtää 3D-näkymän. trimesh tekee 3D-kappaleet ja .obj-tiedoston. pytest ajaa testit.
requirements.txt vko 41	kirjastolista. Tiedosto, jossa ovat projektin kirjastot versioineen. Kirjastot asennetaan komennolla <code>pip install -r requirements.txt</code> . Uusi kirjasto lisätään tänne vain ohjaajan luvalla.
tagi vko 41	Nimetty versio Gitissä, esimerkiksi v0.0.41, v0.1 tai v1.0. Tagin push käynnistää julkaisun.
GitHub Actions vko 41	GitHubin automaatio. Kun pushaat tagin, automaatio ajaa testit ja tekee Windows-version. Sitten se ajaa Windows-versiolle itsetestin ja lisää zipin releaseen.

paketointi vko 41	Sovellus ja sen kirjastot kootaan kansioksi, joka toimii ilman Pythonia. Tässä projektissa paketoinnin tekee PyInstaller tiedoston vektoripaja.spec ohjeilla.
itsetesti vko 41	Sovelluksen oma tarkistus: Vektoripaja.exe --itsetesti. Se lukee esimerkki-SVG:n ja tekee siitä 3D-kappaleet ja .obj-tiedoston. Ikkunaa ei avata. GitHub Actions ajaa sen jokaiselle julkaisulle.
release vko 41	julkaisusivu. GitHubin sivu, josta valmiin version voi ladata. Tässä projektissa releasessa on zip, jossa on Windows-versio.
zip vko 41	pakattu kansio. Yksi tiedosto, jonka sisällä on kansio. Pura zip ennen käyttöä: hiiren oikea painike ja Pura kaikki.
main vko 41	päähaara. Repositoryn päälinja. Julkaisu tehdään päähaarasta.

kahden yrityksen sääntö vko 41	Kun sama asia epäonnistuu kahdesti, lopetetaan yrittäminen ja palataan Copilotille.
täydennys vko 43	GitHub Copilotin harmaa koodiehdotus VS Codessa. Hyväksytään Tab-näppäimellä. Ei kuluta krediittejä.
moduuli vko 43	Kansio tai tiedosto, jolla on yksi vastuu. Esimerkiksi tuonti on eri moduuli kuin vienti.
havaintoissue vko 43	label havainto. Issue, johon kirjataan virhe: mitä odotit, mitä tapahtui ja miten virheen saa toistettua. Suljetaan korjauscommitilla.
layer vko 43	Inkscapen taso. Inkscapen piirroksen taso. Layerin nimestä tulee mallin osan nimi.

solmupuu vko 44	vanhempi-lapsi-puu. Mallin rakenne, jossa jokainen osa on solmu. Solmulla on nimi, lapset ja oma muunnos. Kun vanhempi liikkuu, lapset liikkuvat mukana, koska lapsen paikka lasketaan vanhemman kautta.
maailmamuunnos vko 44	Osan lopullinen paikka, kierto ja koko 3D-näkymässä. Lasketaan näin: vanhemman maailmamuunnos kertaa osan oma muunnos.
hierarkiapaneeli vko 44	Sovelluksen paneeli, joka näyttää solmupuun osat sisennettyinä.
puhdas funktio vko 44	Funktio, joka ei muuta mitään itsensä ulkopuolella. Sama syöte antaa aina saman tuloksen.
rajapinta vko 44	Funktion nimi, syöte ja paluuarvo. Rajapinta kirjoitetaan korttiin ennen toteutusta.

revolve vko 45	pyörähdyskappale. Puoliprofiili pyörähtää akselin ympäri, ja siitä syntyy kappale, esimerkiksi maljakko.
orbit vko 45	Näkymän kierto hiirellä mallin ympäri.
kysymystila vko 45	Ask. GitHub Copilotin tila, joka vastaa kysymyksiin mutta ei muuta tiedostoja.
inflate vko 46	polusta putki. Viivapolusta tulee putki, jossa on 3–8 sivua. Esimerkiksi johto tai sarvi.
orientaatiowidget vko 46	kameran suuntakuvio. Pieni akselikuvio näkymän kulmassa. Kun klikkaat sen akselia, kamera kääntyy katsomaan mallia suoraan akselin suunnasta.
suora näkymä vko 46	Katsot mallia tasan yhden akselin suunnasta, kuin pöydän reunalta tai ylhäältä. Suunnitelmassasi tämä on osa Orthographic Plane Snapia.

ortografinen näkymä vko 46	Näkymä, jossa kaukana olevat osat eivät pienene kuten valokuvassa. Tehdään tärkeässä jatkossa. Pakollisessa ytimessä kaukana olevat osat näyttävät pienemmiltä, ja se on oikein.
view lock vko 46	Painike, joka estää kameran kiertymisen, kun teet tarkkaa työtä.
transformi vko 47	Osan siirto, kierto ja skaalaus.
transformipaneeli vko 47	Paneeli, jossa osan siirto, kierto, skaalaus ja pivot syötetään numerokenttiin X, Y ja Z. Toimii myös näppäimistöllä.
pivot vko 48	Piste, jonka ympäri kappale kiertää ja skaalautuu. Oletuksena keskipisteessä.
OBJ vko 49	.obj-tiedosto. 3D-tiedostomuoto. Jokainen osa alkaa rivillä, jonka alussa on o-kirjain.

virheenkorjausketju vko 49	Havainto → toistamisohje → syy → korjauscommit → uusintatesti → regressiotesti. Projektissa tehdään kolme ketjua.
regressiotesti vko 49	Testi, joka toistaa korjatun virheen ja jää testeihin. Se varmistaa, ettei virhe palaa.
vikatehtävä vko 49	Ohjaajan tekemä tarkoituksellinen virhe. Siitä tehdään virheenkorjausketju, jos aitoa havaintoa ei ole.
JSON vko 50	tallennusmuoto. Tekstimuotoinen tiedosto, johon projektin tiedot tallennetaan ja josta ne avataan uudelleen.
tietoturva-arvio vko 50	Taulukko: uhka, testi, tulos ja toimenpide. Tässä projektissa uhkia ovat esimerkiksi haitallinen SVG ja rikottu JSON.
katselmointi vko 51	Asiakas kokeilee versiota ja kertoo, mitä muutetaan. Asiakkaan sanat kirjataan erillään omasta tulkinnasta.

haara vko 3	Oma työlinja. Muutokset eivät vaikuta päähaaraan, ennen kuin haara liitetään.
pull request vko 3	muutospyyntö. Muutospyyntö, jolla haara liitetään päähaaraan. Luet muutokset ennen liittämistä.
merge vko 3	Haaran liittäminen päähaaraan.
kahva vko 5	Tartuntakohta 3D-näkymässä. Kahvasta vetämällä osaa siirretään, kierretään tai skaalataan.
ruudunlukija vko 5	Lukija. Ohjelma, joka lukee näytön sisällön ääneen. Windowsin oma ruudunlukija on Lukija (Narrator). Se käynnistyy ja sammuu näppäimillä Ctrl + Windows-näppäin + Enter.

Accessibility Insights vko 5	Accessibility Insights for Windows. Microsoftin ilmainen työkalu, joka tarkistaa Windows-sovelluksen ikkunan saavutettavuuden. Sen pikatarkistus FastPass listaa löydettyt virheet, esimerkiksi painikkeen, jolta puuttuu nimi.
RC vko 6	julkaisuehdokas, release candidate. Versio, jossa sisältö on jäädytetty ja joka testataan ennen v1.0:aa. Tagi on v1.0-rc1.
sisältöjäädytys vko 6	Uusia ominaisuuksia ei enää lisätä. Vain estävät virheet korjataan.
työnäyte vko 7	Yksi tuotos, joka osoittaa osaamisesi: commit, issue, testi, kuva tai muistio.
näyttömatriisi vko 7	Tämän sivun lista tutkinnon vaatimuksista. Jokaisella rivillä on linkki työnäytteeseen.

Näyttömatriisi – 31

osaamisvaatimusta

Rasti vasta, kun vaatimukselle on täsmällinen työnäyte: linkki, commit, kuva, testirivi tai muistio. Sama työnäyte voi kelvata useaan kohtaan.

Ohjelmointi

- **käyttää ohjelmointieditoria tai kehitysympäristöä** – VS Code, GitHub Copilot, Pythonin virtuaaliympäristö ja pytest käytössä viikoilta 40 ja 41. Versiot README:ssä. Paluuviikon ajo viikolla 2.
- **etsii ja korjaa virheitä ohjelmakoodista** – Kolme virheenkorjausketjua havaintoissueista viikoilla 49, 4 ja 7: havainto, toistamisohje, syy omin sanoin, korjauscommit, uusintatesti ja regressiotesti.
- **testaa ohjelman toimintoja** – Testit 1–26 ja lisätestit viikoilta 41, 43–50 ja 3–5, myös maailmamuunnoksen lisätesti (44). Kirjoitat odotetun tuloksen itse testikoodiin ennen toteutusta. Tulokset issueissa.
- **käyttää rakenteista ohjelmointia toteutuksissa** – Kolme funktiota selityspohjalla: hierarkian muunnos (viikko 44), pivotin syötteen tarkistus (48) ja OBJ-objektijako (49). Lisäksi maailmamuunnos puhtaana

funktiona (44). Rajapinta ennen toteutusta. Selitys ääneen seuraavan viikon palaverissa.

☐ **tulkitsee suunnitelmia ja toteuttaa käyttöliittymän tai sen osia** – Hierarkiapaneeli (44), view lock (46), transformipaneeli (47–48) sekä isot kahvat ja näppäimistökäyttö (5) käyttöliittymävaatimuksesi mukaan. Accessibility Insights ennen ja jälkeen.

☐ **tulkitsee suunnitelmia ja toteuttaa ohjelmiston toimintoja** – MVP-kuvauksesta tehtäväkortit ja toiminnot: tuonti (43), revolve (45), inflate (46) ja transformit (47) hyväksymiskriteerien mukaan.

☐ **sopii tehtävistä tiimin muiden jäsenten kanssa** – Issue-kommentit "Sovittu viikkopalaverissa pp.kk." joka viikolta 41–7.

☐ **etsii ratkaisuvaihtoehtoja ja ratkoo ongelmia yhdessä tiimin kanssa** – Revolven valintatavan vertailu ja yhteinen päätös (44–45). Osien tunnistustapa Päivitä SVG -toimintoon (3).

☐ **arvioi ratkaisujen toimivuuden yhdessä tiimin kanssa** – Katselmointi asiakkaiden kanssa (51). Palaverin arvio MVP:n ratkaisusta tärkeää jatkoa varten (2).

☐ **arvioi omaa toimintaa tiimin jäsenenä** – Itsearviointi kolmesta tilanteesta viikolla 9. Se

lähetetään ohjaajalle Teamsissa. Päiväkirjaan kirjataan vain tilanteiden päivät ja issue-numerot.

Ohjelmistokehittäjänä toimiminen

- ☐ **selvittää kehitystiimin kanssa asiakkaan tarpeet** – Kysymyslista asiakkaille (40). Vastaukset tiedostossa project-docs/kysymykset.md (41). MVP omin sanoin.
- ☐ **viestii tekniset asiat asiakaslähtöisesti** – Viiden minuutin demo katselmoinnissa (51). README ja käyttöohje käyttäjälle (6).
- ☐ **osallistuu version katselmointiin** – project-docs/katselmointi.md: asiakkaiden sanat, oma tulkinta ja sovitut muutokset (51).
- ☐ **asettaa kehitystiimin kanssa toteutettavat toiminnot tärkeysjärjestykseen** – Karsinta pakolliseen ytimeen, tärkeään jatkoon ja jatkolistaan perusteluineen (40). Asiakkaiden prioriteetit (51). Tärkeän jatkon järjestys palaverissa (2).
- ☐ **jakaa kehitystiimin kanssa toteutettavat toiminnot tehtäviksi** – Tehtäväkortit hyväksymiskriteereineen viikolta 41 alkaen. Oma rajausehdotus ja tarkistusrivi issueissa.
- ☐ **suunnittelee ja arvioi kehitystiimin kanssa tehtävien toteuttamista** – Kaistan valinta ja toteuman

vertailu Raportoi-askeleessa. Krediittimittaus (40).

Tärkeän jatkon tuntiarviot (2).

□ **kehittää ohjelmiston toimintalogiikkaa** – Hierarkia, revolve, inflate, transformit ja pivot (44–48) sekä Päivitä SVG (3).

□ **valitsee ohjelmistoon sopivan tietovaraston** – Tallennustapojen vertailu oman suunnitelman kriteereillä ja oman mallin JSON-tiedoston koko (50).

□ **toteuttaa yhteyden tietovarastoon** – Tallennus ja avaus valitulla tallennusmuodolla. Versionumero ja rakenteen tarkistus (50).

□ **hyödyntää rajapintoja ja käsittelee tietoa** – Tiedoston avaus QFileDialogilla ja SVG:n luku svgelementsillä (43), .obj-vienti trimeshillä (49) ja JSON (50).

□ **arvioi ohjelmiston tietoturvaa** – Testi 4: haitallinen SVG (43). Testi 22: rikottu tallennus (50). Tietoturva-arvio project-docs/tietoturva.md: uhka, testi, tulos ja toimenpide.

□ **käyttää versionhallintaa** – Commitit rivillä Closes #N koko projektin ajan. Tagit: harjoitus v0.0.41, viikkoversiot v0.0.43–v0.0.49, v0.1, v1.0-rc1 ja v1.0.

□ **liittää ohjelman osan olemassa olevaan versioon** – Päivitä SVG omassa haarassa, pull request ja merge

päähaaraan, kun kaikki testit menevät läpi (3).

□ **julkaisee ohjelman tuotantoympäristöön** –

Release GitHub Actionsilla: Windows-versio, itsetesti ja zip. Harjoitus (41), viikkoversiot (43–49), MVP v0.1 (50) ja v1.0 (7).

Ohjelmiston toteuttaminen ohjelmistokomponenttikirjastolla

□ **ottaa käyttöön ja konfiguroi**

ohjelmistokomponenttikirjaston käyttöön soveltuvan kehittämisympäristön – PySide6 + PyVista + trimesh + pytest -pohja omin komennoin: virtuaaliympäristö, kirjastojen asennus ja tarkista_ymparisto.py (41).

□ **selvittää ohjelmistokomponenttikirjaston tarjoamat mahdollisuudet ja rajoitteet** – project-docs/kirjastot.md: svgelements (43) ja putkigeometrian (46) rajoitteet omilla tiedostoilla kokeiltuina.

□ **käyttää ohjelmistokomponenttikirjaston tärkeimpiä toimintoja ja työkaluja** – trimeshin revolve, PyVistan tube-suodatin, kamera ja orientaatiowidget sekä Qt:n transformipaneeli (45–48).

□ **tuo kehittämisympäristöön ulkoisia**

komponentteja – pip-paketit omin komennoin ja requirements.txt (41).

- **suunnittelee, toteuttaa ja testaa ohjelmiston ohjelmistokomponenttikirjastoa käyttäen** – MVP v0.1 komponenttikirjastolla testeineen (43–50).
- **julkaisee ohjelmiston asiakkaan ympäristöön** – MVP-release asiakkaiden kokeiltavaksi (50). Antti vahvistaa v1.0-releasen omalla Windows-koneellaan, Matti demon tai opiskelijan koneen kautta (7).
- **dokumentoi ohjelmiston sovitulla tavalla** – Dokumentointitapa sovittu palaverissa (43). README ja käyttöohje, joiden avulla julkaisutestaaja kulkee koko polun (6).

Muista: sivuston rastit ja kentät tallentuvat vain selaimeen. Ne eivät siirry opettajalle eivätkä korvaa Gitissä olevaa työtä.